

Alysson Cirilo Silva

Template de monografia da UFMA em L^AT_EX

São Luís-MA

2025

Alysson Cirilo Silva

Template de monografia da UFMA em $\text{\LaTeX}$

Monografia apresentada ao curso de Ciência da Computação da Universidade Federal do Maranhão, como parte dos requisitos necessários para obtenção do grau de Bacharel em Ciência da Computação.

Universidade Federal do Maranhão

Orientador: Francisco José da Silva e Silva

São Luís-MA

2025

Ficha gerada por meio do SIGAA/Biblioteca com dados fornecidos pelo(a) autor(a).
Núcleo Integrado de Bibliotecas/UFMA

Silva, Alysson Cirilo.

Descoberta e desconexão de objetos inteligentes smart
objects em ambientes oportunisticos de IoMT / Alysson
Cirilo Silva. - 2019.

40 f.

Orientador(a): Francisco José da Silva e Silva.

Monografia (Graduação) - Curso de Ciência da
Computação, Universidade Federal do Maranhão, São Luís,
2019.

1. IoMT. 2. IoT. 3. Middleware. I. Silva, Francisco
José da Silva e. II. Título.

Alysson Cirilo Silva

Template de monografia da UFMA em L^AT_EX

Monografia apresentada ao curso de Ciência da Computação da Universidade Federal do Maranhão, como parte dos requisitos necessários para obtenção do grau de Bacharel em Ciência da Computação.

Trabalho aprovado. São Luís-MA, 19 de Julho de 2025:

Francisco José da Silva e Silva
Doutor — UFMA

Carlos de Salles Soares Neto
Doutor — UFMA

Marcelo Henrique Monier Alves
Júnior
Mestre — IFMA

São Luís-MA
2025

Agradecimentos

Em primeiro lugar, agradeço pela dádiva da vida e pela força para superar os desafios ao longo desta jornada.

À instituição de ensino e a todos os seus professores e colaboradores, pela oportunidade de acesso a uma educação de qualidade e pelo conhecimento compartilhado, que foi fundamental para o meu crescimento.

Ao meu orientador, pela confiança, paciência, e por todo o aprendizado proporcionado. Agradeço também pela oportunidade de participar de projetos enriquecedores e desafiadores, que certamente marcaram minha trajetória.

À minha família, pelo apoio incondicional, amor e compreensão em cada etapa. Sem vocês, nada disso seria possível.

Aos membros da banca examinadora, pela dedicação e por contribuírem com suas análises e sugestões para o aprimoramento deste trabalho.

Aos colegas de jornada, pelo companheirismo e por tornarem esta caminhada mais leve e significativa.

A todos os amigos que estiveram presentes ao longo do caminho, compartilhando desafios e alegrias, meu mais sincero agradecimento. Obrigado.

*The road to wisdom? Well, it's plain
And simple to express:
Err
and err
and err again,
but less
and less
and less.
(Piet Hein)*

Resumo

Com a crescente demanda por formatos padronizados e a importância de facilitar o processo de formatação, a utilização de templates em $\text{\LaTeX}$ para a elaboração de monografias tem se tornado uma prática comum em diversas instituições acadêmicas. No contexto da UFMA, a adoção de um *template* específico para monografias, alinhado ao padrão ABNT, visa garantir a padronização da estrutura e a conformidade com as normas da instituição e da Associação Brasileira de Normas Técnicas. Esse template facilita a organização dos conteúdos e a inserção de elementos como referências bibliográficas, tabelas e figuras. Este trabalho tem como objetivo apresentar e detalhar o uso de um template em $\text{\LaTeX}$ para a elaboração de monografias na UFMA, abordando suas principais funcionalidades e vantagens no processo de formatação conforme as diretrizes da ABNT.

Palavras-chave: ABNT. UFMA.

Abstract

With the growing demand for standardized formats and the importance of simplifying the formatting process, the use of $\text{\LaTeX}$ templates for writing theses has become a common practice in various academic institutions. In the context of UFMA, the adoption of a specific template for theses, aligned with ABNT standards, aims to ensure the standardization of structure and compliance with the institution's and the Brazilian Association of Technical Standards (ABNT) guidelines. This template helps organize content and insert elements such as bibliographic references, tables, and figures. This work aims to present and detail the use of a $\text{\LaTeX}$ template for writing theses at UFMA, highlighting its main features and advantages in the formatting process according to ABNT guidelines.

Keywords: ABNT. UFMA.

Lista de ilustrações

Figura 1 – Diagrama de sequência do processo de entrega de mensagens no MQTT 20

Lista de tabelas

Tabela 1	–	Exemplo	21
Tabela 2	–	Dados de leitura por país	21
Tabela 3	–	Performance	21
Tabela 4	–	Uso de aspas no $\text{\LaTeX}$	25
Tabela 5	–	Especificando unidades de medida	29

Sumário

	Sumário	10
1	INTRODUÇÃO	12
1.1	Sobre esse <i>template</i>	12
1.2	Por onde começo?	12
1.2.1	Tenho experiência com $\text{\LaTeX}$	12
1.2.2	Não tenho experiência com $\text{\LaTeX}$	14
1.3	Fluxo de trabalho	14
1.3.1	Escrevendo: <code>just continuous</code>	14
1.3.2	Validando: <code>just build</code>	14
1.3.2.1	Suprimindo avisos do ChkTeX	15
1.3.2.2	Adicionando palavras ao dicionário	15
1.3.3	Workflows alternativos	15
1.4	Organização do texto	16
2	ABNT	17
2.1	Alíneas	17
2.2	Citações	17
2.2.1	Citação indireta	17
2.2.2	Citação direta	18
2.3	Figuras	19
2.4	Tabelas	20
2.5	Estrangeirismos	21
3	MISC	22
3.1	Código	22
3.1.1	Linguagens e estilos	23
3.2	Remissões internas	24
3.2.1	<i>Labels</i> em capítulos, seções e subseções	24
3.2.2	<i>Labels</i> em figuras	25
3.2.3	<i>Labels</i> em tabelas	25
3.2.4	<i>Labels</i> em códigos	25
3.3	Aspas	25
3.4	Termos muito usados no seu documento	26
3.4.1	Plural e capitalização	27
3.4.2	Termos aninhados	29

3.5	Unidades de medida	29
3.5.1	Unidades	29
3.5.2	Quantidades	30
3.6	Fonte	30
	Referências	31

1 Introdução

1.1 Sobre esse *template*

Esse projeto é um *template* “opinionado” de como estruturar e organizar um trabalho de conclusão de curso—focado em Ciência da Computação—utilizando a classe `abntex2`.

O projeto somente oferece um ponto de partida para criação de documento, com configurações e inclusões de pacotes que o autor considera úteis, assim sendo, é recomendada a leitura da documentação oficial da classe `abntex2`. O projeto também vem configurado com automação para compilação com `latexmk` ou `just`, e com sistema de CI/CD no GitHub. Como diria Knuth (1974), devemos nos preocupar com clareza antes de performance.

1.2 Por onde começo?

1.2.1 Tenho experiência com $\text{\LaTeX}$

Se você já tem familiaridade com $\text{\LaTeX}$ e quer um resumo de como usar esse *template*, você pode olhar o código fonte desse documento e entender como montar sua monografia. Essas informações irão te auxiliar:

- a) edite o arquivo `metadata.tex` com os dados do seu trabalho;
- b) altere o arquivo de ficha catalográfica localizada em:
`content/pre-textual/ficha-catalografica/index.pdf`, com o documento correspondente ao seu trabalho;
- c) o arquivo `monografia.tex` é o arquivo principal, ele inclui os arquivos no diretório `content/`;
- d) o *template* foi construído de forma que cada seção fica em um subdiretório, agrupado pela sua função no documento segundo a ABNT: as partes pré-textuais (resumo, agradecimentos, epígrafe, etc.) ficam em `content/pre-textual/` e os capítulos de desenvolvimento em `content/textual/`, ambos seguindo o padrão `content/{grupo}/{nome-da-seção}/index.tex`:
 - lembre-se de atualizar o arquivo `monografia.tex` ao renomear o diretório ou o arquivo;
 - sintá-se à vontade para mudar a organização, como por exemplo:

`content/textual/section-01/index.tex`.

- e) termos que aparecem frequentemente no texto podem ser movidos para macros no arquivo `macros.tex` para garantir grafia e formatação consistentes;
- f) a inclusão de pacotes e as configurações de preâmbulo têm o `tccconfig.sty` como ponto de entrada, que carrega módulos temáticos no diretório `preamble/` (fontes, bibliografia, `minted`, `hyperref` e macros) para facilitar a manutenção;
- g) as referências bibliográficas estão em `bib/biblio.bib`.

O documento é compilado com $\text{Xe}\text{L}\text{A}\text{T}\text{E}\text{X}$ com o auxílio do programa `latexmk`. O projeto inclui um `justfile` (a configuração do `just`), que oferece uma interface mais simples de compilação; os seguintes comandos são suportados:

- a) `just pdf`: compila o projeto e gera o PDF (receipe padrão, equivalente a rodar `just` sem argumentos);
- b) `just continuous`: compila o projeto e gera o PDF, adicionalmente, fica executando e compila novamente cada vez que um arquivo do projeto muda;
- c) `just format`: formata todos os arquivos fonte (`.tex`, `.sty`, `.cls` e `.bib`) do projeto com o `latexindent`;
- d) `just format-check`: verifica se os arquivos fonte estão formatados, sem alterá-los, e falha caso algum precise ser formatado (usado na CI);
- e) `just lint`: roda o *linter* `ChkTeX` sobre os arquivos `.tex`, apontando problemas comuns de $\text{L}\text{A}\text{T}\text{E}\text{X}$ (espaçamento, balanceamento de chaves, uso de `\cite`, etc.) e falha caso encontre algum (usado na CI);
- f) `just spell`: roda o corretor ortográfico `hunspell` sobre o texto dos arquivos `.tex` (ignorando os comandos $\text{L}\text{A}\text{T}\text{E}\text{X}$), usando os dicionários `pt_BR` e `en_US` do sistema, e falha caso encontre palavras desconhecidas (usado na CI). Termos válidos que o dicionário não reconhece (siglas, nomes próprios, termos técnicos) devem ser adicionados, um por linha, ao arquivo `dictionary.txt`;
- g) `just check`: roda `just format-check`, `just lint` e `just spell` em sequência, validando o projeto sem compilá-lo nem alterar arquivos;
- h) `just build`: executa `just check` e, se passar, compila o PDF com `just pdf`;
- i) `just clean`: deleta os arquivos auxiliares de compilação;
- j) `just cleanall`: deleta os arquivos auxiliares de compilação e o PDF gerado;
- k) `just help`: lista todas as receitas disponíveis com uma breve descrição de cada uma.

O projeto está configurado para separar os arquivos gerados do código fonte:

- a) **output/**: diretório onde o PDF final é gerado;
- b) **build/**: diretório onde ficam os arquivos auxiliares de compilação. Esses são arquivos temporários que o $\text{\LaTeX}$ cria durante o processo de compilação (índices, referências cruzadas, logs de erro, etc.) e não precisam ser versionados ou editados manualmente.

Também deve ser possível compilar o projeto com o *build system* ou IDE de sua preferência.

1.2.2 Não tenho experiência com $\text{\LaTeX}$

Vai ter que aprender o básico até entender o que está descrito na [subseção 1.2.1](#). Esses recursos podem ser bons pontos de partida:

- a) [wiki do abntex](#);
- b) [tutorial do Overleaf](#);
- c) [wikibook](#).

1.3 Fluxo de trabalho

Esta seção descreve como usar o projeto no dia a dia, desde a escrita até a validação final antes de versionar as alterações.

1.3.1 Escrevendo: **just continuous**

Durante a escrita, o comando mais útil é o **just continuous**: ele compila o PDF e recompila automaticamente a cada alteração salva, mostrando o resultado quase em tempo real. Para encerrar, interrompa o processo (**Ctrl+C**).

Para aproveitar bem esse fluxo, use um visualizador de PDF que detecte alterações no arquivo e atualize a exibição sozinho (por exemplo, Zathura, Okular, Skim ou Evince). Caso contrário, será preciso reabrir ou atualizar o PDF manualmente a cada mudança.

1.3.2 Validando: **just build**

Antes de um *commit*, é uma boa prática rodar o **just build**: ele executa o **just check** (**format-check**, **lint** e **spell**) e, se tudo passar, compila o PDF. Como são os mesmos passos da CI, rodá-lo localmente evita quebrar o *pipeline*. Para só validar sem gerar o PDF, use **just check**; para formatar automaticamente, use **just format**.

As verificações que mais costumam exigir intervenção são o *linter* (**just lint**) e o corretor ortográfico (**just spell**), tratados a seguir.

1.3.2.1 Suprimindo avisos do ChkTeX

O **just lint** usa o ChkTeX, um *linter* que aponta problemas comuns de L^AT_EX (espaçamento, balanceamento de chaves, uso de aspas, etc.). Cada problema tem um identificador numérico, mostrado na saída do **just lint** (por exemplo, “Warning 26”), e é com esse número que o aviso é suprimido.

A maioria dos avisos vale a pena corrigir, mas às vezes o ChkTeX acusa um falso positivo: uma construção proposital que dispara o aviso. Um exemplo é escrever um emoticon, em que o espaço antes do “:” vira o aviso 26 (espaço antes de pontuação) e o “)” vira o aviso 10 (parêntese de fechamento solto). Para silenciar avisos, basta um comentário **% chktex N** no fim da linha, encadeando vários se necessário:

```
1 % Dispara os avisos 26 e 10
2 Fiquei muito feliz :)
3
4 % Comentario no fim da linha silencia ambos
5 Fiquei muito feliz :) % chktex 26 % chktex 10
```

Código 1 – Supressão de avisos do ChkTeX

O **% chktex N** precisa ficar no fim da linha, pois o **%** inicia um comentário e tudo após ele é descartado. Para desligar um aviso em todo o projeto, adicione o número ao bloco **CmdLine** do **.chktxrc** (por exemplo, **-n26**), mas prefira a supressão por linha, para não perder o aviso onde ele é legítimo.

1.3.2.2 Adicionando palavras ao dicionário

O **just spell** usa o hunspell com os dicionários **pt_BR** e **en_US** do sistema. Siglas, nomes próprios e termos técnicos não reconhecidos são reportados como palavras desconhecidas e fazem a verificação falhar. Quando a palavra estiver correta, adicione-a (uma por linha) ao **dictionary.txt**, que guarda os termos válidos do projeto.

Note que esses dicionários vêm do sistema operacional e a versão pode variar entre a sua máquina e a CI, então uma palavra aceita localmente pode ser apontada como desconhecida na CI (ou vice-versa). Mantê-la no **dictionary.txt** resolve a divergência, já que o arquivo é versionado e vale para todos os ambientes.

1.3.3 Workflows alternativos

Manter o **dictionary.txt** é um custo recorrente que nem todo mundo vai querer pagar. Se a verificação ortográfica não fizer sentido para o seu fluxo, há alternativas:

- a) remover o **spell** da recipe **check** no **justfile**, mantendo formatação e *linter*, mas não o corretor;
- b) remover de vez a recipe **spell** do **justfile** e o passo correspondente na CI (`.github/workflows/ci.yaml`);
- c) ignorar as validações e usar só o **just pdf** (ou **just continuous**) para compilar.

O mesmo vale para o **format-check** e o **lint**: são um ponto de partida, e cada autor pode ajustar o **justfile** e a CI ao seu gosto.

1.4 Organização do texto

O restante deste documento mostra como utilizar o *template* para as coisas mais comuns na monografia. Ao longo do texto, algumas citações são inseridas propositalmente em diferentes páginas para demonstrar o comportamento das referências bibliográficas (como o backref e a contagem de citações).

2 ABNT

Este capítulo mostra como usar os macros específicos do pacote `abntex2`. Isso é apenas um resumo, consulte a documentação oficial.

2.1 Alíneas

Alíneas são listas, devemos usar o ambiente `alíneas`, desta forma:

```

1 \begin{alíneas}
2   \item primeira alínea;                % termina com ;
3   \item segunda alínea                  % termina com :
4   \begin{alíneas}
5     \item primeira subalínea da segunda alínea; % termina com ;
6     \item última subalínea da segunda alínea.  % termina com .
7   \end{alíneas}
8   \item última alínea.                  % termina com .
9 \end{alíneas}

```

Código 2 – Alíneas

O texto que vem antes, geralmente, deve terminar com ‘:’, desta forma: O [Código 2](#) vai gerar o seguinte:

- a) primeira alínea;
- b) segunda alínea
 - primeira subalínea da segunda alínea;
 - última subalínea da segunda alínea.
- c) última alínea.

2.2 Citações

O *template* utiliza `biblatex` com o estilo `abnt` para gerenciar as referências do artigo, elas devem ser salvas no arquivo `bib/biblio.bib`, a sintaxe se assemelha ao que está descrito no [Código 3](#).

2.2.1 Citação indireta

Para fazer uma citação indireta, é apenas necessário usar a macro `\cite`, passando o id da referência. Como apresentado no [Código 4](#).

```

1 @book{knuth:aocp:1997,
2   author = {Knuth, Donald E.},
3   title = {The art of computer programming, volume 1 (3rd ed.): fundamental
4     ⇨ algorithms},
5   year = {1997},
6   isbn = {0201896834},
7   publisher = {Addison Wesley Longman Publishing Co., Inc.},
8   address = {USA}
9 }
10 @article{dijkstra:goto:1968,
11   title={Go To Statement Considered Harmful},
12   author={Dijkstra, Edsger W},
13   journal={Communications of the ACM},
14   volume={11},
15   number={3},
16   pages={147--148},
17   year={1968},
18   publisher={ACM New York, NY, USA}
19 }

```

Código 3 – Sintaxe BibTeX

```

1 Programar pode ser bastante prazeroso~\cite{knuth:aocp:1997}.

```

Código 4 – Citação indireta

O resultado vai ser: Programar pode ser bastante prazeroso (KNUTH, 1997).

Também pode-se usar a macro `\textcite` para adicionar o autor sem parênteses, como no Código 5.

```

1 De acordo com \textcite{knuth:goto:1974}, otimização prematura é a raiz de todo
  ⇨ mal.

```

Código 5 – Citação com textcite

O resultado vai ser: De acordo com Knuth (1974), otimização prematura é a raiz de todo mal.

2.2.2 Citação direta

Use o ambiente `citacao` como no Código 6.

O resultado será:

```

1 \begin{citacao}
2   For a number of years I have been familiar with the observation that
3   the quality of programmers is a decreasing function of the density of go
4   to statements in the programs they produce. More recently I
5   discovered why the use of the go to statement has such disastrous
6   effects, and I became convinced that the go to statement should be
7   abolished from all \enquote{higher level} programming
   ↪ languages.~\cite{dijkstra:goto:1968}.
8 \end{citacao}

```

Código 6 – Citação direta

For a number of years I have been familiar with the observation that the quality of programmers is a decreasing function of the density of go to statements in the programs they produce. More recently I discovered why the use of the go to statement has such disastrous effects, and I became convinced that the go to statement should be abolished from all “higher level” programming languages. (DIJKSTRA, 1968).

Como o texto em questão está em inglês, deveríamos usar a opção **english**, desta forma: `\begin{citacao}[english]`. O que vai deixar o texto em itálico:

For a number of years I have been familiar with the observation that the quality of programmers is a decreasing function of the density of go to statements in the programs they produce. More recently I discovered why the use of the go to statement has such disastrous effects, and I became convinced that the go to statement should be abolished from all “higher level” programming languages. (DIJKSTRA, 1968).

2.3 Figuras

Use o ambiente **figure** como no Código 7, que produz a Figura 1.

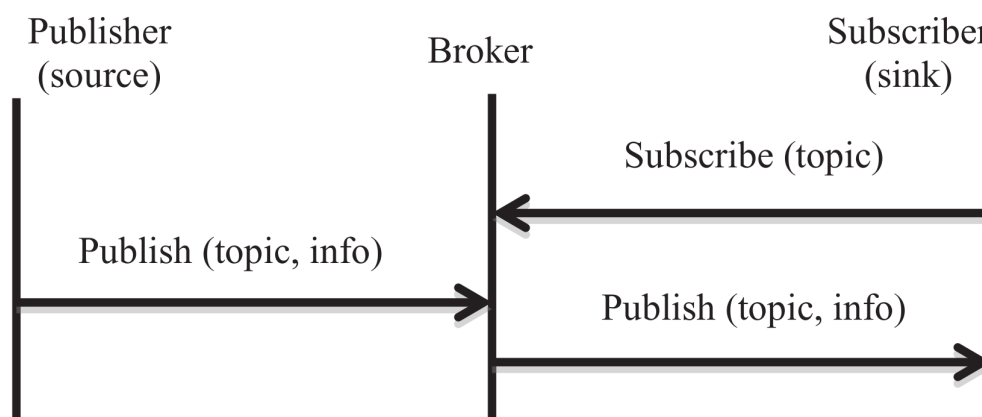
```

1 \begin{figure}[htb]
2   \caption{Diagrama de sequência do processo de entrega de mensagens no \mqtt}
3   \label{fig:mqtt}
4   \centering
5   \includegraphics[width=0.85\linewidth]{\CurrentFilePath/res/img/mqtt-sequence.png}
   ↪ g}
6   \fonte{\textcite{al-fuqaha:et-al:2015}}
7 \end{figure}

```

Código 7 – Adição de imagem

Figura 1 – Diagrama de sequência do processo de entrega de mensagens no MQTT



Fonte: Al-Fuqaha *et al.* (2015)

2.4 Tabelas

Tabelas no padrão ABNT têm uma sintaxe complicada, copie e cole o [Código 8](#) altere os dados, ele vai gerar a [Tabela 1](#).

```

1 \begin{table}[htb]
2   \begin{center}
3     \IBGEtab{
4       \caption{Exemplo}
5       \label{tab:fake}
6     }{
7       \begin{tabular}{lcr} % left, center, right
8         \toprule
9         \textbf{Col 1} & \textbf{Col 2} & \textbf{Col 3} \\
10        \midrule
11        Texto          & Outro texto   & Mais texto \\
12        1.1            & $\log_2 n$    & 3.14 \\
13        \bottomrule
14      \end{tabular}
15    }{
16      \fonte{\autoriaproprias}
17    }
18   \end{center}
19 \end{table}
  
```

Código 8 – Tabela

Você vai encontrar outras tabelas nesse capítulo, consulte o código fonte desse documento caso queira ver como são construídas.

Tabela 1 – Exemplo

Col 1	Col 2	Col 3
Texto	Outro texto	Mais texto
1.1	$\log_2 n$	3.14

Fonte: Produzido pelo autor

Tabela 2 – Dados de leitura por país

País	Livros lidos por ano (média)	Gênero mais popular
Brasil	4,7	Ficção
Estados Unidos	10	Ficção
Japão	12	Mangás
Alemanha	12	Ficção
França	21	Romances
Índia	4,7	Ficção

Fonte: ChatGPT :)

Tabela 3 – Performance

	Média	Desvio padrão	Mínimo	Máximo	Intervalo de 95% de confiança
Δt (ms)	20.1	3.7	7.0	78.9	20.1 ± 0.03

Fonte: Produzido pelo autor

2.5 Estrangeirismos

Utilize o comando `\foreign` para deixar o termo em itálico. Tome como exemplo o [Código 9](#), ele gerará o seguinte parágrafo, com a palavra “*dataset*” em itálico:

Para a análise estatística, foi necessário utilizar um *dataset* abrangente, contendo uma grande quantidade de dados que permitiram gerar insights significativos sobre o comportamento do usuário.

1 Para a análise estatística, foi necessário utilizar um `\foreign{dataset}`
↪ abrangente, contendo uma grande quantidade de dados que permitiram gerar
↪ insights significativos sobre o comportamento do usuário.

Código 9 – Estrangeirismos

Não existe uma regra, mas entende-se que algumas palavras já sofreram aportuguesamento e não necessitam de grafia especial, o [manual da SECOM](#) pode ser usado como referência.

3 Misc

3.1 Código

Frequentemente você vai precisar mencionar pequenos trechos de código no meio de um parágrafo, como o nome de um comando, de um arquivo ou de uma variável. Para isso, o comando `\texttt` (de *typewriter text*) formata o seu argumento com uma fonte monoespaçada, sinalizando ao leitor que aquilo é código. Por exemplo, `\texttt{int x = 0;}` produz `int x = 0;`. Repare, porém, que o `\texttt` apenas troca a fonte: ele não conhece a linguagem do trecho e, portanto, não aplica realce de sintaxe.

Quando o trecho inline for código de verdade e o realce de sintaxe ajudar na leitura, prefira o comando `\mintinline`, fornecido pelo pacote `minted`. Ele recebe a linguagem como primeiro argumento e o código delimitado por um par de caracteres iguais à sua escolha (por exemplo, barras verticais). Assim, escrever `\mintinline{java}|int x = 0;|` produz `int x = 0;`. Use o `\texttt` para menções curtas e simples (nomes de comandos, arquivos e identificadores) e reserve o `\mintinline` para trechos de código reais em que o realce agregar clareza.

Para trechos maiores, que merecem destaque próprio e uma legenda, use o ambiente `minted` dentro de um float `listing`. Como lembra Knuth (1974), programas devem ser escritos para humanos lerem. Utilize conforme o Código 10.

```

1 \begin{listing}[htb]
2   \begin{minted}{java}
3     public class Main {
4       public static void main(String[] args) {
5         System.out.println(factorial(10));
6       }
7
8       static int factorial(int n) {
9         if (n == 1) return 1;
10
11         return n * factorial(n - 1);
12       }
13     }
14   \end{minted}
15   \caption{Fatorial}
16   \label{lst:factorial-inline-output}
17 \end{listing}

```

Código 10 – Inclusão de código

O conteúdo dentro do ambiente `minted` será colocado dentro do seu documento, como os exemplos de código que você viu até agora, no caso do [Código 10](#), este iria gerar como output o [Código 11](#).

```

1 public class Main {
2     public static void main(String[] args) {
3         System.out.println(factorial(10));
4     }
5
6     static int factorial(int n) {
7         if (n == 1) return 1;
8
9         return n * factorial(n - 1);
10    }
11 }

```

Código 11 – Fatorial

Caso queira deixar seu código `LaTeX` mais compacto, você pode colocar o seu código em um arquivo separado e incluir em seu documento com o comando `\inputminted`. Esse *template* sugere que coloque esse códigos em `content/{grupo}/{nome-da-seção}/res/code`. Assumindo que o código existe em um arquivo `Factorial.java` dentro desse diretório, o exemplo anterior poderia ser escrito conforme o [Código 12](#).

```

1 \begin{listing}[htb]
2     \inputminted{java}{\CurrentFilePath/res/code/Factorial.java}
3     \caption{Fatorial}
4     \label{lst:factorial-include-output}
5 \end{listing}

```

Código 12 – Inclusão de código através de um arquivo externo

3.1.1 Linguagens e estilos

A linguagem do código é informada como argumento do ambiente `minted` (o *lexer* do Pygments, como `java`, `latex` ou `python`), o que dá o realce correto de palavras reservadas, strings e comentários sem precisar cadastrá-las à mão. O tema de cores é global e definido no módulo do `minted` (`preamble/minted.sty`, carregado pelo `tccconfig.sty`), com o comando `\usemintedstyle`; opções padrão (moldura, numeração, fonte) ficam no `\setminted`. Consulte a documentação oficial para coisas mais complexas.

3.2 Remissões internas

Se você quer referenciar partes do seu documento, como: capítulos, seções, subseções, figuras, tabelas e códigos, você deve primeiramente adicionar uma *label* nesse elemento, depois deve usar o comando `\autoref` ou `\ref` no local que deseja mencioná-lo.

O comando `\autoref` adiciona um link para o elemento contendo o nome que o identifica (como: “capítulo”, “seção”, “subseção”, “tabela” e etc.) seguido do número desse elemento no documento.

O comando `\ref` adiciona um link para o elemento contendo *apenas* o número do elemento no documento.

Como exemplo, adicionei a *label* “`sec:cross-ref`” na seção atual, ao utilizar o comando `\autoref{sec:cross-ref}`, vou obter o seguinte resultado: “[seção 3.2](#)”. Entretanto, se utilizar `\ref{sec:cross-ref}`, vou obter apenas: “[3.2](#)”, que pode ser útil ao referenciar múltiplos elementos ao mesmo tempo.

A forma de adicionar *labels* é ligeiramente diferente dependendo do tipo de elemento sendo referenciado.

3.2.1 *Labels* em capítulos, seções e subseções

Para capítulos, seções e subseções, usamos o comando `\label` seguido do id da *label* que queremos atribuir, conforme o [Código 13](#).

```
1 \chapter{Introdução} \label{chap:intro}
2
3 Blá.
4
5 \section{Panorama Geral} \label{sec:overview}
6
7 Blá.
8
9 \subsection{Questões Fundamentais} \label{sub:fundamental-questions}
10
11 Blá.
```

Código 13 – Atribuição de *labels* em capítulos, seções e subseções

Desta forma, eu posso referenciar o capítulo “Introdução” usando o comando: `\autoref{chap:intro}`.

3.2.2 *Labels* em figuras

Também se utiliza o comando `label`, entretanto, dentro do ambiente `figure` e após a macro `caption`, consulte o [Código 7](#).

3.2.3 *Labels* em tabelas

Também se utiliza o comando `label`, entretanto, dentro do primeiro bloco do comando `IBGETab` e após a macro `caption`, consulte o [Código 8](#).

3.2.4 *Labels* em códigos

Assim como em figuras e tabelas, usamos o comando `label` dentro do float `listing` (criado pelo `minted`), logo após a macro `caption`, consulte os códigos [10](#) e [12](#).

3.3 Aspas

Aspas em $\text{\LaTeX}$ são mais complicadas do que parecem. Ao tentar usar os caracteres “'” ou “”” (aspas simples e dupla, respectivamente), o resultado não será o esperado, não gerando as aspas curvas. O uso errôneo desse caracteres no [Código 14](#) irá gerar o seguinte resultado: ‘Aspas simples’ e ”aspas duplas”.

```
1 'Aspas simples' e "aspas duplas".
```

Código 14 – Uso incorreto de aspas

Note que “'” sempre gera aspas direitas e “”” sempre gera aspas duplas retas. Para produzir aspas corretamente, o $\text{\LaTeX}$ espera que se utilize os símbolos presentes na [Tabela 4](#).

Tabela 4 – Uso de aspas no $\text{\LaTeX}$

Resultado desejado	Símbolo	Descrição do símbolo	Resultado
Aspa simples esquerda	`	1 acento grave	‘
Aspa simples direita	'	1 aspa simples	’
Aspa dupla esquerda	``	2 acentos grave	“
Aspa dupla direita	''	2 aspas simples	”

Fonte: Produzido pelo autor

Ou seja, o exemplo anterior deveria ser escrito conforme o [Código 15](#), gerando: ‘Aspas simples’ e “aspas duplas”.

```
1 'Aspas simples' e ``aspas duplas''.
```

Código 15 – Uso correto de aspas

Como esse método de inclusão de aspas é pouco intuitivo e fácil de errar, podemos usar o pacote `csquotes` para auxiliar. Usamos os comando `enquote*` para aspas simples e `enquote` para aspas duplas. O exemplo pode ser então reescrito conforme o [Código 16](#), gerando: ‘Aspas simples’ e “Aspas duplas”.

```
1 \enquote*{Aspas simples} e \enquote{Aspas duplas}.
```

Código 16 – Uso do pacote csquotes

3.4 Termos muito usados no seu documento

Se o seu texto utiliza com bastante frequência termos que você quer que tenham tipografia e escrita consistente, elas podem ser adicionadas como macros no arquivo `macros.tex`, com o comando `newterm`. Vamos olhar alguns exemplos no [Código 17](#).

```
1 \newterm{\api}{API}
2 \newterm{\broadcast}{\foreign{broadcast}}
3 \newterm{\printf}{\texttt{printf}}
```

Código 17 – Definição de macros

Neste caso, foram definidos 3 macros (`\api`, `\broadcast` e `\printf`), a intenção do autor é que o termo API sempre seja escrito em maiúsculo, o termo *broadcast* sempre seja escrito como uma palavra estrangeira e o termo `printf` sempre seja formatado com fonte monoespaçada.

Para usar os macros definidos pelo autor, basta incluí-los no texto desejado, conforme o [Código 18](#), gerando o seguinte resultado:

Usando essa API, você pode fazer um *broadcast* de dados para vários dispositivos. Para verificar o status, pode-se utilizar `printf` para imprimir mensagens no console.

Repare nas chaves vazias (`\broadcast{}`) usadas quando o termo é seguido de um espaço. Como qualquer comando L^AT_EX sem argumentos, um termo “engole” o espaço que vem logo depois dele: escrever `\broadcast de` produziria “*broadcastde*”, sem o espaço. As chaves vazias delimitam o fim do comando e preservam o espaço seguinte. Quando o termo

```

1 Usando essa \api, você pode fazer um \broadcast{} de dados para vários
  ↪ dispositivos. Para verificar o status, pode-se utilizar \printf{} para imprimir
  ↪ mensagens no console.

```

Código 18 – Uso de macros

é seguido de pontuação — como em `\api`, — as chaves não são necessárias. O projeto também roda um linter (via `just lint`), que alerta sobre termos seguidos de espaço sem as chaves.

Como alternativa às chaves vazias, a sequência `\broadcast\ de dados` (barra invertida seguida de espaço) também preserva o espaço após o termo, gerando: *broadcast de dados*.

3.4.1 Plural e capitalização

Um mesmo termo costuma aparecer no singular, no plural e com a primeira letra maiúscula (no início de uma frase, por exemplo). Em vez de criar uma macro para cada variação, o `newterm` aceita chaves opcionais e dois modificadores de uso, conforme o [Código 19](#).

```

1 \newterm{\vetor}{vetor}[plural=vetores]
2 \newterm{\framework}{\foreign{framework}}[plural=\foreign{frameworks}]
3 \newterm{\justica}{justiça}
4 \newterm{\id}{id}[plural=ids, cap=ID, capplural=IDs]

```

Código 19 – Definição de termos com plural e capitalização

A opção `plural` define a forma do termo no plural. Sem ela, o termo não tem plural e o modificador `[s]` interrompe a compilação — útil para termos incontáveis, como `\justica` no [Código 19](#).

As opções `cap` e `capplural` permitem fornecer formas capitalizadas explícitas, necessárias quando a capitalização automática não produz o resultado desejado. Quando omitidas, a primeira letra do singular e do plural é capitalizada automaticamente, preservando qualquer formatação (a capitalização automática atravessa comandos como `\foreign` ou `\texttt` e capitaliza apenas a primeira letra do texto). O termo *framework*, definido no `macros.tex`, ilustra o caso automático preservando o itálico:

- a) `\framework` produz “*framework*”;
- b) `\framework*` produz “*Framework*”;
- c) `\framework[s]` produz “*frameworks*”;

d) `\framework*[s]` produz “*Frameworks*”.

Já o termo `\id` do [Código 19](#) precisa das opções, a forma automática seria “Id” e “Ids”, mas `cap` e `caplural` fixam as siglas corretas “ID” e “IDs”.

No uso, o asterisco (*) produz a versão capitalizada e o argumento opcional `[s]` produz a versão no plural; os dois podem ser combinados. O comando ainda age como um validador estrito: pedir o plural de um termo que não o define interrompe a compilação. O [Código 20](#) reúne esses casos.

```

1 \vetor      % vetor
2 \vetor[s]   % vetores
3 \vetor*     % Vetor
4 \vetor*[s]  % Vetores
5 \framework % framework (em itálico)
6 \framework* % Framework (capitaliza preservando o itálico)
7 \justica    % justiça
8 \justica*   % Justiça
9 \justica[s] % quebra a build: \justica não define plural
10 \id        % id
11 \id[s]     % ids
12 \id*       % ID (sem cap, sairia "Id")
13 \id*[s]    % IDs (sem caplural, sairia "Ids")

```

Código 20 – Uso dos modificadores de plural e capitalização

O asterisco também pode ser usado na *definição* do termo, com um papel diferente: `\newterm*` cria um termo cuja capitalização fica *bloqueada*, útil para nomes que nunca devem começar com maiúscula. O [Código 21](#) define dois termos assim: `\iphone`, cujo “i” minúsculo faz parte da marca, e `\pdfextension`, uma extensão de arquivo.

```

1 \newterm*{\iphone}{iPhone}[plural=iPhones]
2 \newterm*{\pdfextension}{\texttt{.pdf}}

```

Código 21 – Definição de termos com capitalização bloqueada

Com isso, `\iphone` e `\iphone[s]` produzem “iPhone” e “iPhones” normalmente, mas `\iphone*` interrompe a compilação com um erro, impedindo a forma capitalizada. Da mesma forma, passar as opções `cap` ou `caplural` a um termo criado com `\newterm*` é contraditório e falha já na definição. Esse rigor evita que variações inexistentes passem despercebidas no texto final.

3.4.2 Termos aninhados

O texto de um termo pode conter outros termos, inclusive com seus próprios modificadores de plural e capitalização. No [Código 22](#), o termo `\tabelaponteiros` reaproveita o termo `\ponteiro` na definição do singular e do plural.

```
1 \newterm{\ponteiro}{ponteiro}[plural=ponteiros]
2 \newterm{\tabelaponteiros}{tabela de \ponteiro[s]}[plural=tabelas de \ponteiro[s]]
```

Código 22 – Definição de termos aninhados

A capitalização aninhada também funciona: `\tabelaponteiros` resulta em “tabela de ponteiros”, `\tabelaponteiros[s]` em “tabelas de ponteiros” e `\tabelaponteiros*` em “Tabela de ponteiros”, capitalizando apenas a primeira letra e preservando o termo interno.

3.5 Unidades de medida

Aconselho o uso do pacote `siunitx`, a leitura do manual é extremamente útil para descobrir todos os comandos que você vai precisar usar. Aqui está um resumo:

3.5.1 Unidades

Use `unit` para escrever unidades de medida. Esse comando aceita duas sintaxes, uma usando macros predefinidas, e outra usando strings. Por exemplo, para escrever “metros por segundo ao quadrado”, podemos usar `\unit{\meter\per\second\squared}` ou `\unit{m/s^2}`. Entretanto, esses dois comandos vão produzir resultados levemente diferente, conforme

Tabela 5 – Especificando unidades de medida

Comando	Resultado
<code>\unit{\meter\per\second\squared}</code>	m s^{-2}
<code>\unit{m/s^2}</code>	m/s^2

Fonte: Produzido pelo autor

Apesar de representar a mesma unidade, por padrão, o macro `\per` usa o formato de exponencial, isso pode ser alterado passando um argumento opcional, da seguinte forma: `\unit[per-mode=symbol]{\meter\per\second\squared}`, gerando: m/s^2 .

Também existem alguns macros abreviados, no caso anterior, poderíamos escrever `\unit[per-mode=symbol]{\m\per\s\squared}`, para obter: m/s^2 .

Os prefixos como quilo, mega, e giga também estão definidos e podem ser usados normalmente, para obter kg, usa-se `\unit{\kilo\gram}`.

Prefixos binários também estão disponíveis (kibi, mebi, gibi, etc), assim como macros para bit e byte, por exemplo: `\unit[per-mode=symbol]{\mebi\byte\per\second}`, produz MiB/s.

3.5.2 Quantidades

Quantidades podem ser especificadas com a macro `qty`, essa macro aceita dois parâmetros obrigatórios, o número e a unidade. Para especificar “5 gibibytes”, você usaria `\qty{5}{\gibi\byte}`, que produz: 5 GiB.

3.6 Fonte

A ABNT não determina um tipo de fonte específico, mas é um equívoco comum que deve ser utilizado a fonte Arial. Se você precisar mudar a fonte, esse documento é compilado com $\text{\LaTeX}$ e usa o pacote `fontspec`, que permite usar qualquer fonte instalada no seu computador através do comando `\setmainfont{Nome da Fonte}`.

Referências

- DIJKSTRA, E. W. Go To Statement Considered Harmful. *Communications of the ACM*, ACM New York, NY, USA, v. 11, n. 3, p. 147–148, 1968. Citado 2 vezes na página 19.
- AL-FUQAHA, A. *et al.* Internet of things: A survey on enabling technologies, protocols, and applications. *IEEE communications surveys & tutorials*, IEEE, v. 17, n. 4, p. 2347–2376, 2015. DOI: [10.1109/COMST.2015.2444095](#). Citado 0 vez na página 20.
- KNUTH, D. E. Structured Programming with go to Statements. *Computing Surveys*, ACM, v. 6, n. 4, p. 261–301, 1974. DOI: [10.1145/356635.356640](#). Citado 3 vezes nas páginas 12, 18, 22.
- KNUTH, D. E. *The art of computer programming, volume 1 (3rd ed.): fundamental algorithms*. USA: Addison Wesley Longman Publishing Co., Inc., 1997. ISBN 0201896834. Citado 1 vez na página 18.